

第九章 软件安全设计

9.1 安全设计原则

- 原则1：让最弱的环节变得安全
- 首先使得安全性最薄弱的环节变得安全。安全实践者通常认为：安全是一个链条，其安全性由最弱的地方决定。软件的安全由最弱的组件决定。
- 攻击者选择系统中最弱的地方攻击，因为那里最容易攻破。攻击者最可能攻击一个软件系统中的一个弱点，而不是试图穿透一个坚固的防线。



原则2：实践纵深防御

常识：多道防线使银行更安全

- 银行金库的多道防线



原则2：实践纵深防御

- 纵深防御的思想是通过采用多个不同的防御策略来管理风险，因此，某一层防御不够充分的时候，另一层将阻止系统被攻破。
- 纵深防御有助于减少系统的单一失效点，这种分层的安全防御措施能够减少攻击成功的概率。
- 例如除了采取登录认证的手段之外，还可以采取基于权限的访问控制技术，防止攻击者在获取了他人的登录口令之后在系统中任意操作。

原则3：安全的错误退出

- 有很多系统发生各种错误时会表现出不安全的行为。这样，攻击者只需要导致一个错误，或者等待错误的发生，然后就可以利用系统在错误处理时的不安全行为发起攻击。
- 由于**错误是无法避免的**，应该事先计划的是当错误发生时哪些安全问题需要避免。
- 任何复杂的系统都应该有错误模式，当系统发生错误时，系统应该能够安全地退出。

问题：哪个是正确的登录程序？

1

```
DWORD dwRet = IsAccessAllowed(...); //权限判断的返回值
if (dwRet == ERROR_ACCESS_DENIED) {
    // Security check failed.
    // Inform user that access is denied.
} else {
    // Security check OK.
}
```

2

```
DWORD dwRet = IsAccessAllowed(...);
if (dwRet == NO_ERROR) {
    // Secure check OK.
    // Perform task.
} else {
    // Security check failed.
    // Inform user that access is denied.
}
```

例子：不安全的错误退出

```
DWORD dwRet = IsAccessAllowed(...);  
if (dwRet == ERROR_ACCESS_DENIED) {  
    // Security check failed.  
    // Inform user that access is denied.  
} else {  
    // Security check OK.  
}
```

这个处理程序的缺陷在于当IsAccessAllowed()函数返回错误时，可能有多种情况。若发生的情况不在判定范围内，例如系统内存耗尽ERROR NOT ENOUGH MEMORY，用户可能执行一个特权任务。

原则4：最小权限原则

- 仅仅将所需权限的**最小集授权**给需要访问资源的主体，并且该**权限的持续时间应该尽量短**。
- 给用户不必要的（过多的）权限通常导致信息被泄漏和不必要的改变。因此，细致地访问权限分配将减少对系统的破坏。

原则5：权限分离原则

需要两把钥匙开一把锁比使用一把钥匙要更加安全可靠。



又如银行的共管账户

原则5：权限分离原则

- 系统在授权访问某个对象时应该确保多个条件得到满足，通过检查单一条件来决定是否访问对强安全性来说不够充分。
- 如果攻击者能够获得一个特权但没有第二个，将不能发起有效的攻击。
- 如果软件系统主要由一个组件组成，那么使用多种检查措施访问不同组件的想法将无法实现。则将软件分离成多个组件且需要多次访问检查将阻止攻击，或有效地避免对整个系统的一次性攻陷。

原则6：保护机制的简单性

- 评价系统安全时的一个因素是复杂性
- 如果设计、实现、或者安全机制非常复杂，那么安全弱点存在的可能性将会增加。复杂系统中的精细的问题可能很难发现。
- 使用公共的功能函数能减少代码数量，程序设计者和开发人员都应该设法使系统尽量简化
- 尽可能地考虑重用组件，只要被重用的组件是高质量的。越是成功的应用了组件，那么写的代码就越少
- 这种思想特别适用于密码库。被广泛使用的密码库要比组织内部使用的密码库更加健壮。当然，即使是被广泛使用的组件，也总有可能出现问题。

原则7：最小共享机制

- 每个共享机制（特别是共享变量）代表用户之间一种潜在的信息路径，这些路径一定要小心处理以确保不会损害安全。
- 避免多个主体通过共享机制有权使用同一个资源，敏感信息可能通过这种机制在不同用户之间共享。
- 例如，互联网上服务应用程序同时允许攻击者和用户获得访问权。

原则8：勉强信任原则

- 开发者应该认为他们系统所在的环境是不安全的。
- 无论是外部系统、代码、人员等等，都应该被时刻关注。
- 开发人员应该设想未知用户输入的所有可能情况，即使用户是已知的，也要怀疑其可能发起攻击。
- 例如，即使开发的是公司内部的软件系统，也依然存在SQL注入攻击的可能性，以及用户不小心导致的错误输入，因此也需要加入输入验证功能。

原则9：不要认为你的秘密是安全的

- 不要把秘密隐藏在代码中，必须使用真正的保护机制来保护敏感信息。应该始终假设：攻击者能够获得关于你系统的足够的信息并且发起攻击。
- 例如：汇编和反编译器等可以帮助攻击者获得可能存储在二进制文件中的敏感信息。

```
1 function connectionDatabase(url, userName, password) {  
2     // ....  
3 }  
4  
5 connectionDatabase('./api', 'zhangsan', '1234567');
```

- 必须使用真正的保护机制来保护敏感信息。
- 用户唯一的要做的是保护密钥不会丢失或被窃取。

原则10：完全仲裁（完全控制）

- 当一个主体试图读取一个对象时，操作系统应该仲裁这个行为。
- 首先，决定这个主体能否读这个对象。如果可以，则提供资源。如果这个主体试图再次读这个对象，系统应该再次检查是否能再次读这个对象。
- 大多数系统不会做第二次检查。它们可能把第一次检查的结果放入缓存，第二次访问时基于缓存中的结果。
- 短时间的保存权限信息可以提高系统性能，但是可能产生越权访问的代价。完全仲裁和使用性形成制约关系。

原则11：心理接受能力

- 心理接受能力是指，人们应用的软件**安全设计应该便于使用**，这是一条必不可少的原则。这样，用户会经常正确地使用安全保护机制。当用户想到保护目标机制时，就会不自觉地使用，从而错误就会被减少。
- **安全机制不应该阻止资源的可访问性**，如果安全机制阻碍资源的可用性和可访问性，用户可能选择关闭这些机制。

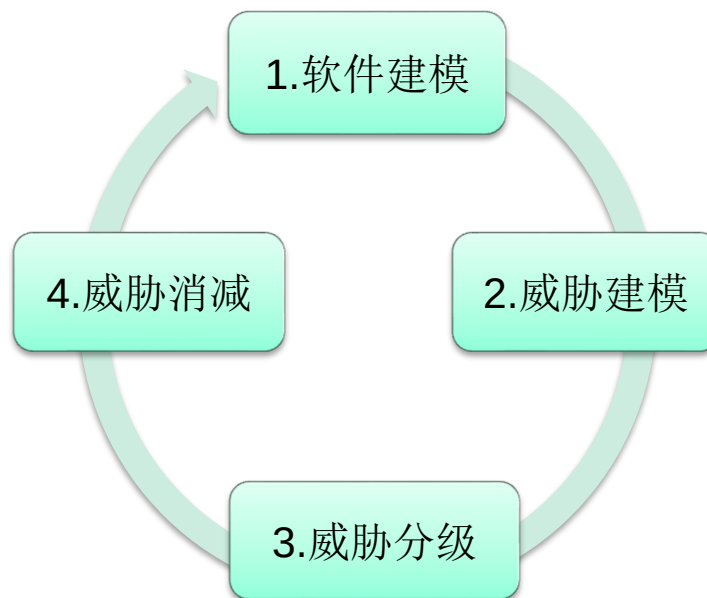
9.2 威胁建模

9.2.1 威胁建模

- 什么是威胁建模？
- 威胁建模以一种**规范化的方式**来考虑应用程序的安全性，是一种基于安全的分析。
- 在软件设计阶段**充分了解各种安全威胁**，有助于人们确定给产品造成的最高安全级别的安全风险是什么，以及攻击是如何表现出来的。
- 创建威胁模型使开发人员认识到产品的什么地方最为危险，因而会选择合适的技术来缓和威胁，这样就使得系统更加安全。

威胁建模的主要步骤

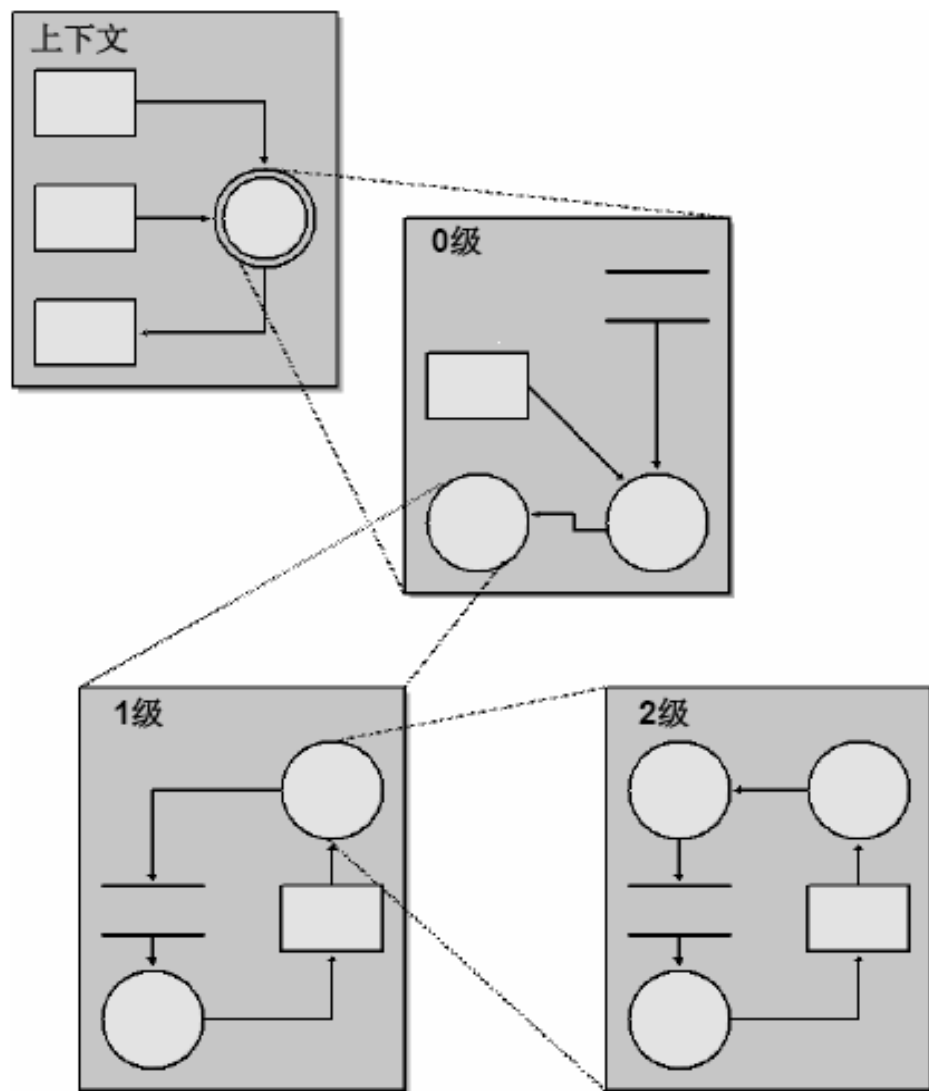
- 1) 分解应用程序，软件建模；
- 2) 确定系统所面临的威胁，威胁建模；
- 3) 以风险递减的方式给威胁排序，威胁分级；
- 4) 选择应付威胁的方法，威胁消减。



1. 软件建模

- 威胁建模需要一种结构化的方法对应用程序进行形式化分解，建立清晰的软件模型有助于寻找威胁。
- 绘制表示软件模型的图表是开始威胁建模非常有效的方法。
- 软件模型有很多种，数据流图和系统结构图是最常用的软件模型。

分解应用程序



2. STRIDE威胁模型

威胁分析就是提出一些安全相关的问题，如

- 未经认证的用户能否浏览秘密的网络数据？
 - 不可信任的用户能否修改数据库中患者的记录数据？
 - 是否有人能够拒绝合法用户获得应用程序的服务？
 - 是否有人能够利用某些特性或组件将自己的特权提升为管理员？
- 没有条理的提出这些问题是否全面呢？

STRIDE威胁模型

- STRIDE建模方法由Microsoft公司提出，该方法通过审查系统设计或架构来发现或纠正设计级的安全问题。
- 假冒 (Spoofing)，典型的例子是使用其他用户的认证信息进行非法访问。
- 篡改 (Tampering)，在未授权的情况下恶意地修改数据。这种修改可以是在数据库中保存的数据，也可能是在网络中传输的数据。
- 否认 (Repudiation)/可抵赖，用户从事一项非法操作，但该用户拒绝承认，且没有方法可以证明她是在抵赖。

威胁模型

- 信息泄漏 (Information disclosure)，信息暴露给不允许对它访问的人。例如用户读到没有给她赋予访问权限的文件的内容，信息在网络中传递时内容被 泄密。
- 拒绝服务 (Denial of service)，拒绝对正当用户的服务。
 -
- 权限提升 (Elevation of privilege)，一个没有特权的用户获得访问特权，从而有足够的权限作出摧毁整个系统的事情。

STRIDE威胁建模的过程

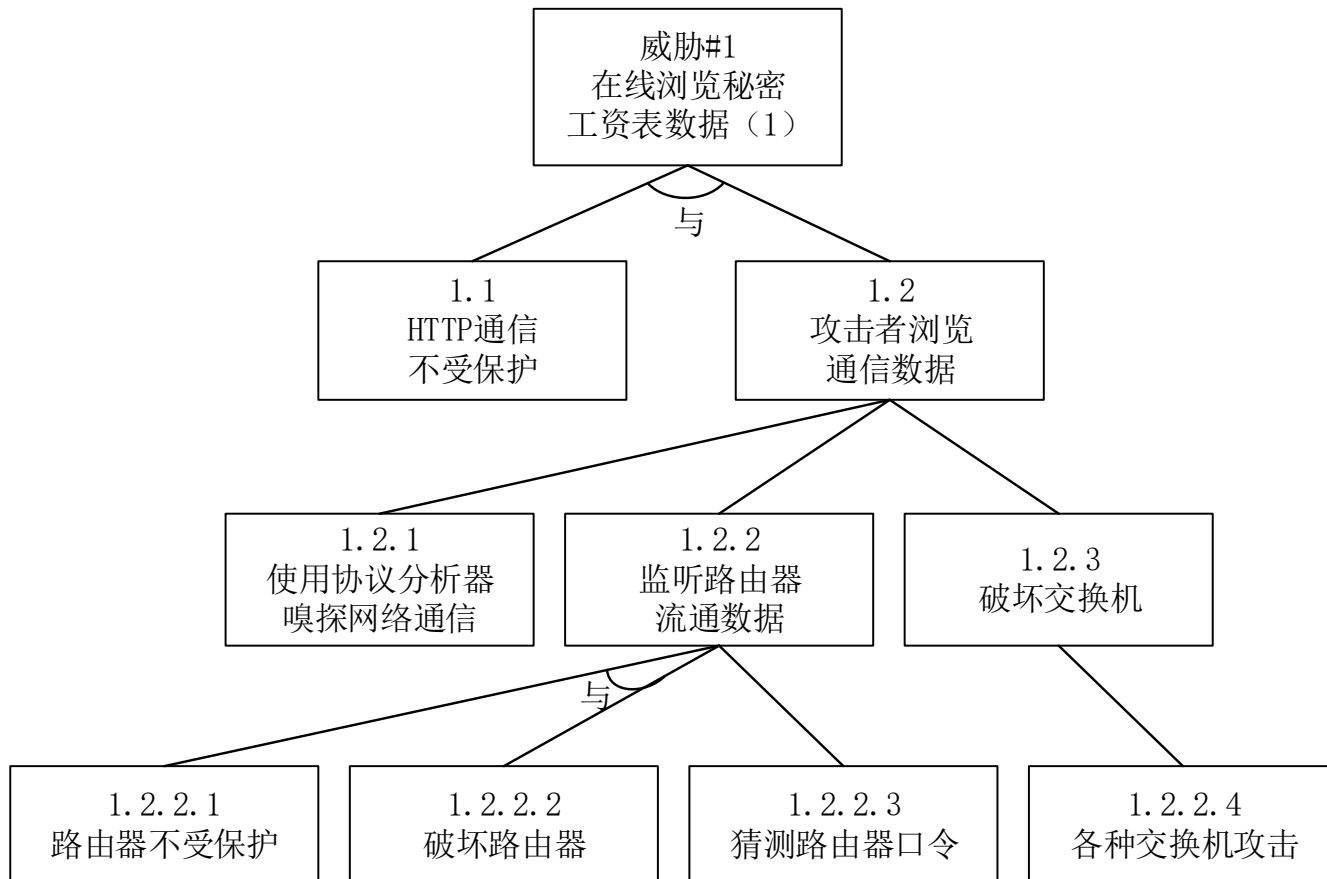
- 到目前为止，将STRIDE模型应用到应用程序中最简单的方法是，考虑模型中的每一个威胁是如何影响每一个组件的，以及是如何影响组件之间的每一个连接或关系的。
- 看一看应用程序的每一部分，为组件或进程判断是否有任何S、T、R、I、D、或者E威胁存在。大部分都会存在许多威胁，将它们都记录下来是很重要的。

威胁的表达方式——建立威胁树

- 建立威胁树，系统地分析所开发的应用程序中的威胁和漏洞。
- 威胁树描述了攻击者破坏各组件所经历的决策过程。
- 以工资数据Web系统为例，用户登录工资数据Web服务器查看自己工资信息。由于工资信息是保密的，这意味着必须保护工资数据的机密性。该信息存在信息泄露的可能，其威胁树

威胁树的示例

工资数据Web服务器的威胁树



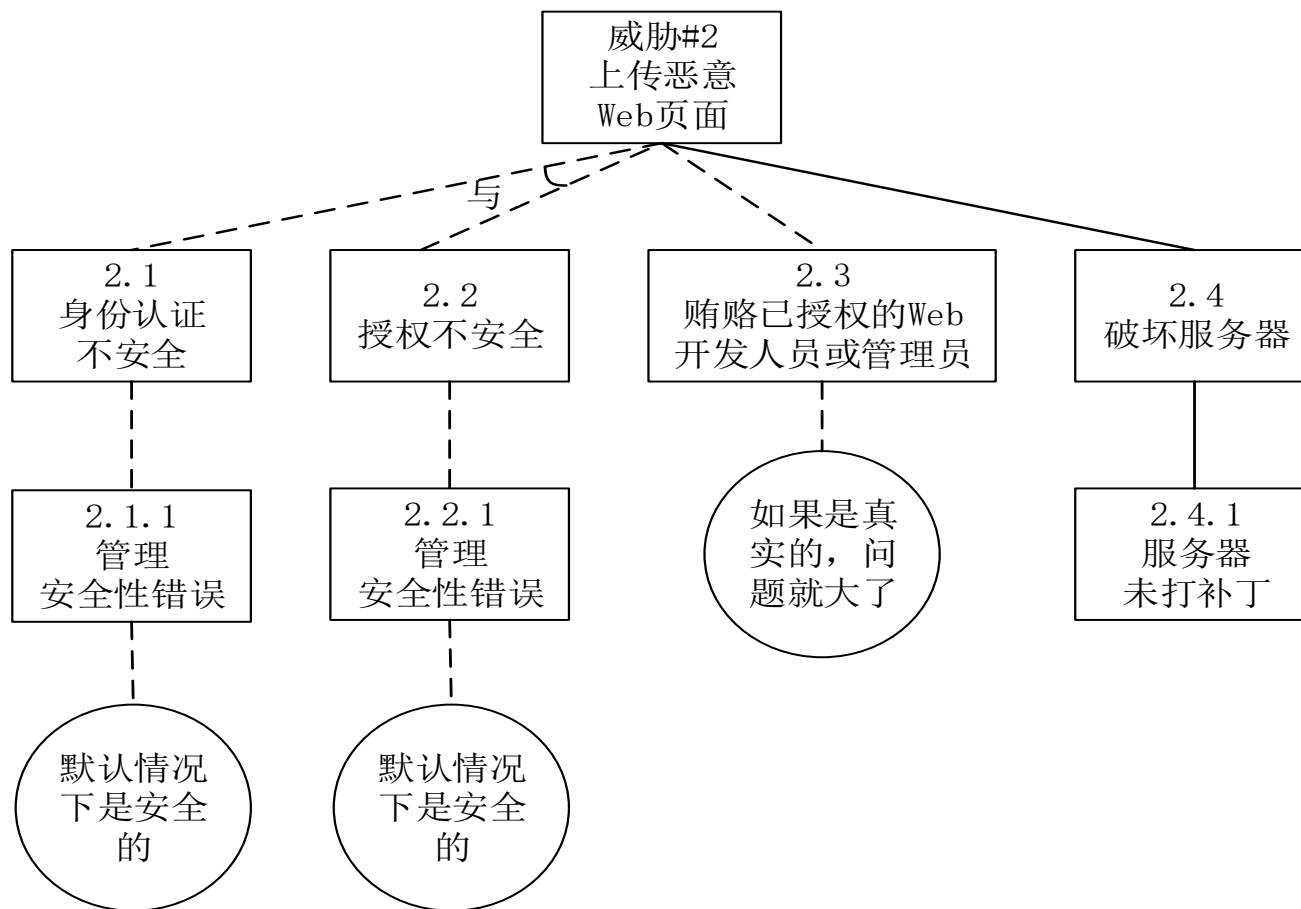
威胁树图

增加威胁树的可读性

- 可以向威胁树添加几个附件，显示最有可能的攻击向量。
- 用虚线表示最不可能的攻击点
- 用实线表示最可能的攻击点
- 在最不可能的攻击节点下放置圆圈，说明攻击为什么会减弱。

威胁树图

图显示了上传恶意Web页面的威胁树，并增加可读性。



3. 威胁分级

当对软件的各个组件进行威胁建模之后，需要对记录的威胁进行分析，**确定各个威胁的等级，从而采取相应的措施减缓威胁**。使用DREAD计算风险

与STRIDE威胁模型类似, DREAD是下面短语的首字母缩写：

- 潜在破坏性 Damage potential
- 再现性 Reproducibility
- 可利用性 Exploitability
- 影响用户 Affected users
- 可发现性 Discoverability

潜在破坏性

潜在破坏性 Damage potential

- 衡量一下威胁可能造成的实际破坏程度, 一般最糟糕的程度(10级)是指让攻击者可能绕开所有的安全限制, 能做任何破坏, 提升特权的威胁通常是10级, 其他的例子与受保护的数据的值有关, 医疗, 金融, 或军事数据通常等级都很高.

再现性

再现性 Reproducibility

- 让潜在的进攻起作用会有多简单?衡量一下将威胁变为结果的简单性,有些bug总是在起作用(10级),而其他的比如复杂的基于时间的竞争状态,却是不可预测的,可能只会偶尔起作用,同样,默认安装的特性中的安全缺陷具有很高的再现性,高再现性对多数攻击来说都是很重要的.

可利用性

可利用性 Exploitability

- 进行一次攻击需要多少努力和专业知识?
- 例如, 如果一个程序员新手使用一台家庭PC进行攻击就能达到目的, 得分就会高达10分, 但是如果需要动用大量人力物力进行一次攻击, 那么得分可能仅仅为1分, 同时, 攻击需要的认证和授权也是需要考虑的, 如果一个匿名远程用户可以攻击, 那么等级为10, 而仅仅本地用户且需要强证书才能进行, 就具有低得多的可利用性.

影响用户

影响用户 Affected users

- 如果威胁被利用并成为一次攻击, 有多少用户受到影响?
- 这可以用攻击影响的用户百分数来粗略的衡量: 100%-91%为10级一直到10%-0%为1级, 尽你所能去估计影响, 服务器和客户机的区别是非常重要的, 服务器会间接影响一大批客户机, 还会潜在的影响其他网络, 还需要考虑的是市场大小和用户的绝对数量, 而不仅仅是百分数, 1亿用户的1%难道还是小数目吗?

可发现性

可发现性 Discoverability

- 这恐怕是最难确定的衡量标准, 最好保险且省事的方式就是, 假设每一个威胁都是可以被发现的, 给每个威胁都定为10级.

威胁分级

- 通过平均这几个数,我们就可以确定DREAD的级别
- 威胁1:恶意用户浏览网络上的秘密工资数据
- 8 潜在破坏性:读取用户的是由工资数据不是开玩笑的!
- 10 再现性:这是100%可再现的。
- 7 可利用性:必须在子网内,或者已经攻破了路由器。
- 10 影响用户:每个人,包括CEO,都会受此影响!
- 10 可发现性:我们就假定它会被发现!
- 风险DREAD : $(8+10+7+10+10)/5=9$
- 显然,9是个很严重的等级,意味着这个威胁应该尽快解决!

4. 威胁消减

确定防御威胁的相关技术。

威胁	威胁消减技术
欺骗	基本认证、Cookie认证、Windows认证、Kerberos认证、PKI、IPSec、数字签名、信息验证码、哈希表
篡改	Windows系统命令、完整性控制、ACLs、数字签名、信息验证码
抵赖	安全日志记录与审计、数字签名、可信的第三方
信息泄露	加密技术、ACLs
拒绝服务	ACLs、过滤处理、授权、高可用性设计
特权提升	ACLs, 组或角色从属关系、特权所有权、许可权、输入验证